Naive RAG: A Practical Guide

Retrieval-Augmented Generation (RAG) combines information retrieval with a language model. Instead of asking a language model to answer only from its learned knowledge, a RAG system first retrieves relevant information from a private knowledge source and then uses that information to generate an answer.

1. Ingestion

Ingestion is the first stage of a RAG pipeline. Documents such as PDFs, text files, or web content are loaded into the system. For a PDF, text can be extracted page by page. The extracted text becomes the raw material for later processing.

2. Chunking

Large documents are divided into smaller pieces called chunks. Chunking makes retrieval more precise because the system can search smaller passages instead of an entire document. A simple rule-based approach can create chunks using a fixed character or word size. Chunking itself does not require an AI model.

3. Embeddings

Each chunk is converted into a numerical vector called an embedding. An embedding model represents the semantic meaning of text in a vector space. Texts with similar meanings tend to have vectors that are close to one another. A common lightweight model is all-MiniLM-L6-v2.

4. Vector Store

The embeddings are stored in a vector index so that similar chunks can be found quickly. FAISS is one example of a vector similarity-search library. The vector index is stored together with the original chunks so that retrieved vector positions can be mapped back to readable text.

5. Query Processing

When a user asks a question, the question is converted into an embedding using the same embedding model used for the document chunks. Using the same embedding space allows the query vector to be compared with stored chunk vectors.

6. Retrieval

The query vector is compared with the vectors in the vector store. The system retrieves the top matching chunks according to a similarity or distance measure. The number of retrieved chunks is commonly controlled with a TOP_K parameter.

7. Reranking

A reranker receives the question and the initially retrieved chunks and scores how relevant each chunk is to the question. A cross-encoder can examine the question and chunk together and produce a relevance score. The results can then be sorted from the highest score to the lowest score. Reranker scores are relative relevance scores, not percentages.

8. Generation

The highest-relevance retrieved chunks are placed into a prompt together with the user's question. A generative language model uses this context to produce the final answer. The goal is to ground the answer in the retrieved information rather than relying only on the model's general knowledge.

9. Complete Naive RAG Flow

A basic pipeline can be represented as: INGEST -> CHUNK -> EMBED -> STORE -> QUERY -> QUERY EMBEDDING -> RETRIEVE -> RERANK -> GENERATE. During ingestion, documents are prepared and stored. During a query, the question is embedded, relevant chunks are retrieved and reranked, and a language model generates an answer using the selected context.

10. Example

Suppose a knowledge document contains information about urban trees. A user asks, 'How do trees help cool cities?' The question is embedded, similar chunks about tree canopies and evaporative cooling are retrieved, a reranker can prioritize the most relevant passages, and the language model can generate an answer using those passages.

11. Why Naive RAG?

Naive RAG is useful for learning because each stage can be implemented and inspected separately. It makes it easier to understand what text was extracted, how chunks were created, which vectors were stored, what was retrieved, how reranking changed the order, and how the final answer was generated.

12. Important Implementation Notes

The vector index and chunk file must come from the same ingestion run. If the PDF is changed, the chunks and vector index should be rebuilt. Otherwise, a query may retrieve text from an older document. It is also useful to keep TOP_K no larger than the number of available chunks.